Búsqueda Semántica de Objetos en Simulación con Razonamiento por Habitaciones

Estado del proyecto y plan de desarrollo

VLMaps + YOLOE + Habitat-Sim · HSSD como dataset principal
Extensión futura: ROS 1 Noetic + Toyota HSR

Mario García Blasco

15 de abril de 2026

Índice

1. Propósito del proyecto **3**
1.1. Problema . . . 3
1.2. Hipótesis . . . 3
1.3. Los tres niveles de conocimiento . . . 3
1.4. Idea clave: razonar sobre la casa, no solo sobre el mapa . . . 3

2. Alcance y decisiones de diseño **3**
2.1. HSSD como dataset principal . . . 3
2.2. MP3D como soporte secundario . . . 4
2.3. Razonamiento por habitaciones como prioridad . . . 4
2.4. Inspiración conceptual: MoMa-LLM . . . 4

3. Estado actual del sistema (14 de abril de 2026) **5**
3.1. Infraestructura — COMPLETADA . . . 5
3.2. Navegación interactiva M2 — COMPLETADA . . . 5
3.3. Mapper LLM — COMPLETADO . . . 5
3.4. YOLOE como verificador — COMPLETADO . . . 5
3.5. Filtros de calidad y heatmap — COMPLETADOS . . . 6
3.6. Integración HSSD — COMPLETADA . . . 6
3.7. Colocación de objetos personalizados — COMPLETADA . . . 6
3.8. Resumen de estado . . . 7

4. Arquitectura funcional actual **7**
4.1. Pipeline de búsqueda M2 . . . 7
4.2. Flujo completo del heatmap . . . 7
4.3. Proveedor de habitaciones HSSD . . . 8
4.4. Archivos clave del proyecto . . . 9

5. Próxima fase: búsqueda con razonamiento por habitaciones **9**
5.1. Visión general . . . 9
5.2. Fase A — Capa de habitaciones sobre HSSD — COMPLETADA . . . 9
5.3. Fase B — Representación de estado por habitación — COMPLETADA . . . 10
5.4. Fase C — Priors objeto → habitación — COMPLETADA . . . 10
5.5. Fase D — Selector de habitación . . . 11

5.6. Fase E — Selección de candidatos intra-habitación 11
5.7. Fase F — Acciones de alto nivel 11
5.8. Fase G — Papel del LLM en la búsqueda 12
5.9. Fase H — Métricas de evaluación 12
5.10. Orden de implementación 13

6. Fase posterior: nuevos objetos y evaluación 13
6.1. Baselines M1 y M3 13
6.2. Evaluación cuantitativa 13
6.3. Objetos adicionales en simulación 14

7. Extensión futura: ROS 1 Noetic + Toyota HSR 14

8. Diario de desarrollo 14

1. Propósito del proyecto

1.1. Problema

Dado un objetivo expresado en lenguaje natural (por ejemplo, *"find the laptop"* o *"ve a la cocina y busca el lavabo"*), un robot simulado debe localizar el objeto de forma eficiente en un entorno interior doméstico. El robot opera en escenas HSSD (*Habitat Static Scene Dataset*) renderizadas con Habitat-Sim. La escena principal es **102344280**, una vivienda de una planta con 15 habitaciones anotadas ($\sim$447 m^2).

El proyecto no busca ampliar infraestructura genérica ni acumular datasets. Su finalidad es construir un sistema de búsqueda inteligente de objetos que entienda la estructura espacial de la vivienda: que razone sobre habitaciones y zonas, que decida dónde buscar según el tipo de objeto y el contexto, que navegue de forma semánticamente guiada y que verifique visualmente el resultado.

1.2. Hipótesis

Un sistema que combine tres niveles de conocimiento — razonamiento a nivel de habitación, memoria semántica de mobiliario (VLMaps) y verificación visual de vocabulario abierto (YOLOE) — encuentra objetos específicos de forma más eficiente que métodos que dependen de un solo nivel de información.

1.3. Los tres niveles de conocimiento

Nivel	Herramienta	Qué sabe
Habitación	`semantic_config.json` (HSSD nativo)	«Esto es la cocina»
Mueble / zona	VLMaps (36 categorías HSSD)	«Hay una encimera allí»
Objeto específico	YOLOE (open-vocab)	«Hay un portátil encima de esa mesa»

1.4. Idea clave: razonar sobre la casa, no solo sobre el mapa

0. El sistema identifica las habitaciones de la escena y selecciona la más probable para el objeto buscado (*room-level reasoning*).
1. Un LLM traduce el objeto a categorías VLMap donde es probable encontrarlo: `laptop` → `[table, counter, shelving]`.
2. VLMaps localiza instancias de esas categorías **dentro de la habitación seleccionada**.
3. El robot navega al candidato más prometedor de esa habitación.
4. YOLOE mira con la cámara y confirma si el objeto está ahí.
5. Si no → siguiente candidato en la habitación, o cambio de habitación. Si sí → **ENCONTRADO**.

2. Alcance y decisiones de diseño

2.1. HSSD como dataset principal

HSSD es un conjunto de escenas de interiores sintéticas de alta calidad desarrolladas para Habitat-Sim. Frente a Matterport3D (MP3D), ofrece:

- **Habitaciones anotadas nativamente**: cada escena incluye un fichero `semantics/scenes/<id>.semanti` con polígonos de planta y nombres de habitación. La clase `SemanticSceneRoomProvider` los rasteriza directamente sobre el grid VLMap sin intervención manual.
- **Geometría limpia e iluminación controlada**: las escenas sintéticas facilitan la verificación visual con YOLOE y eliminan artefactos de reconstrucción fotogramétrica.
- **Alineación con el objetivo del proyecto**: el razonamiento por habitaciones requiere anotaciones de habitaciones fiables. HSSD las proporciona de serie; MP3D requeriría etiquetado manual con LabelMe.

La escena principal es **102344280**: una vivienda de una planta con 15 habitaciones (kitchen, dining room, bedroom ×3, bathroom ×3, hallway, entryway, closet ×3, tv room).

2.2. MP3D como soporte secundario

Toda la funcionalidad MP3D existente se mantiene intacta: el sistema sigue funcionando con escenas MP3D y existe un `LabelMeRoomProvider` listo para usar si se ejecuta el etiquetado manual. Sin embargo, MP3D no condiciona la hoja de ruta actual:

- El etiquetado LabelMe de las 3 escenas MP3D queda **diferido** como tarea secundaria.
- La construcción de VLMaps para escenas MP3D adicionales queda diferida.
- Las limitaciones visuales de MP3D (texturas ruidosas, reconstrucciones imperfectas) perjudican la verificación YOLOE y no son representativas de la capacidad real del método.

2.3. Razonamiento por habitaciones como prioridad

La siguiente gran fase del proyecto es que el robot pueda:

- Recibir instrucciones como *"ve a la cocina"*, *"busca el lavabo"* o *"busca la pica en la cocina"*.
- Identificar las habitaciones de la escena y razonar sobre cuál visitar.
- Decidir, cuando un intento falla, cuál es la siguiente habitación o zona más probable.
- Mantener un estado estructurado de la búsqueda (habitaciones visitadas, candidatos probados, resultados obtenidos).

Esta fase es **prioritaria** frente a la inserción de objetos nuevos en simulación, frente a los baselines M1/M3 y frente a la evaluación cuantitativa. Sin razonamiento por habitaciones, el sistema no puede demostrar la ventaja del nivel de conocimiento más alto de la hipótesis.

2.4. Inspiración conceptual: MoMa-LLM

El paper *Language-Grounded Dynamic Scene Graphs for Interactive Object Search with Mobile Manipulation* (Honerkamp et al., RSS SemRob 2024) propone usar un LLM para razonar sobre un grafo de escena estructurado por habitaciones.

Este proyecto adopta como inspiración conceptual: habitaciones como entidades explícitas, estado estructurado para el LLM, historial compacto de búsqueda y selección iterativa de la siguiente habitación. **No se utiliza ningún código del repositorio de Toyota Motor Europe** (MoMa-LLM). La implementación es completamente propia.

Diferencia principal: MoMa-LLM construye el grafo dinámicamente durante la exploración; este proyecto usa un VLMap pre-construido y habitaciones pre-anotadas (HSSD), lo que permite una evaluación más controlada.

3. Estado actual del sistema (14 de abril de 2026)

3.1. Infraestructura — COMPLETADA

- ✓ Docker con CUDA 12.1 + Habitat-Sim 0.3.1 + conda (Python 3.9)
- ✓ 3 escenas MP3D con VLMaps construidos: **gTV8FGcVJC9_1**, **jh4fc5c5qoQ_1**, **mJXqzFtmKg4_1**
- ✓ 1 escena HSSD integrada: **102344280** (15 habitaciones, ∼447 m²), VLMap construido e indexado
- ✓ Menú interactivo en Docker (`menu.sh`) para toda la pipeline
- ✓ Submodule git `third_party/vlmaps` (rama `tfg-modifications`)
- ✓ Repositorio limpio en GitHub: `mobileclip_blt.ts` (571 MB) eliminado del historial con `git-filter-repo`

3.2. Navegación interactiva M2 — COMPLETADA

- ✓ Pipeline completa: instrucción → LLM → goal selection → visgraph → navegación live con visualización
- ✓ `plan_path_only()`: planificación sin teletransporte; camino mostrado como línea roja 800 ms antes de ejecutar
- ✓ Movimientos lentos y fluidos (`_NAV_STEP_DELAY_MS = 80 ms`)
- ✓ Dirección de cara corregida: $\theta = \arctan 2(-\Delta y, -\Delta x)$
- ✓ Goal selection con 3 niveles de fallback (room centroid / BFS / heatmap peak)
- ✓ Visgraph con margen de 4 px (∼20 cm); recuperación de colisión (giro 30° + replanificación); hasta 3 replanes y 8 candidatos
- ✓ Heatmap semántico overlay (JET colormap, 50 % opacidad)
- ✓ Cámara 1ª persona con inclinación −30
- ✓ Multi-goal retry (hasta 8 candidatos por categoría, por BFS)

3.3. Mapper LLM — COMPLETADO

- ✓ El LLM traduce objetos no-VLMap a categorías VLMap probables. Ejemplo: `laptop` → `[table, counter, shelving]`
- ✓ Distingue queries directas (`sofa` ∈ categorías HSSD) e indirectas (`laptop` ∉ categorías HSSD)
- ✓ Room-based reasoning: sugiere habitaciones probables además de muebles. Ejemplo: `laptop` → `rooms: [office, bedroom]`
- ✓ Prompt con 6 few-shot examples incluyendo rooms

3.4. YOLOE como verificador — COMPLETADO

- ✓ Worker persistente (`_yoloe_persistent_worker.py`): modelo cargado una sola vez; inferencias ∼100–500 ms
- ✓ Protocolo stdout correcto: diagnósticos a `sys.stderr`, `start()` lee hasta recibir `"READY"`
- ✓ `mobileclip_blt.ts` copiado automáticamente al arrancar con búsqueda en múltiples rutas candidatas

- ✓ Centrado visual iterativo (visual servoing): hasta 6 iteraciones de 5° (±15 % tolerancia)
- ✓ Flujo de verificación en 3 stages:
  1. Standoff 0,25 m — YOLOE síncrono
  2. Scan 360° in situ — YOLOE asíncrono (hilo bg)
  3. Componente alternativa del heatmap — YOLOE síncrono
- ✓ 15 categorías YOLOE-incompatibles saltan la verificación (wall, floor, ceiling, beam, railing, etc.)
- ✓ Caché global `get_session(target)`: reutiliza sesión entre instrucciones

3.5. Filtros de calidad y heatmap — COMPLETADOS

- ✓ Filtro de coherencia de heatmaps: mín. 30 celdas, ≥30 % dominancia, ≤20 componentes conexas
- ✓ Filtro por tamaño medio de regiones: descarta contornos <30 % del área media
- ✓ Wall filter: ratio BFS/euclídea > 3× descarta candidatos tras paredes
- ✓ Postprocess heatmap con fórmula de calidad por componente (ver Sección 4.2)
- ✓ Umbral de confianza YOLOE subido de 0,25 a 0,40
- ✓ IndexError boundary corregido con clamp en `plan_path_only()`

3.6. Integración HSSD — COMPLETADA

- ✓ `dataset_type=hssd` activa la rama HSSD sin romper MP3D
- ✓ `scene_dataset_config_file` declarado en Hydra config
- ✓ Navmesh recomputado al iniciar con parámetros correctos
- ✓ 36 categorías HSSD en `get_categories("hssd")`
- ✓ `SemanticSceneRoomProvider`: lee `semantic_config.json` y rasteriza polígonos de habitación sobre el grid VLMap
- ✓ Navegación interactiva validada en escena 102344280

3.7. Colocación de objetos personalizados — COMPLETADA

- ✓ `place_objects.py`: define objetos 3D (.glb) via JSON config
- ✓ Carga automática al iniciar; objetos visibles para YOLOE pero no en VLMap

3.8. Resumen de estado

Componente	Estado
Infraestructura Docker + Habitat + VLMaps	Hecho
Navegación interactiva M2 (BFS + colisiones)	Hecho
Mapper LLM (objeto → cats + rooms)	Hecho
YOLOE persistente + 360° scan + visual servoing	Hecho
Filtros de calidad (coherencia, área, pared)	Hecho
Fórmula de calidad por componente (quality)	Hecho
Integración HSSD completa	Hecho
Colocación de objetos personalizados	Hecho
Fase A: capa de habitaciones sobre HSSD	Hecho
Fase B: `RoomState` + `SearchState`	Hecho
Fase C: priors objeto → habitación	Hecho
Razonamiento por habitaciones (Fases D–H)	**Siguiente prioridad**
Baselines M1 y M3	Posterior
Evaluación cuantitativa	Posterior
Objetos nuevos en simulación	Posterior
Extensión ROS 1 Noetic + Toyota HSR	Futuro

4. Arquitectura funcional actual

4.1. Pipeline de búsqueda M2

El flujo completo del sistema actual es:

1. **Instrucción**: el usuario introduce una query en lenguaje natural.
2. **Parsing LLM**: `parse_object_goal_instruction()` (GPT-4o-mini) extrae categorías semánticas de la instrucción.
3. **Heatmap semántico**: para cada categoría, `compute_heatmap()` genera un mapa de probabilidad 2D a partir de los scores CLIP del VLMap.
4. **Postproceso**: `postprocess_heatmap()` filtra ruido, calcula componentes conexos y asigna un score de calidad a cada componente.
5. **Selección de goal**: se elige el candidato más cercano por coste BFS (M2) o el de mayor calidad (M3, futuro).
6. **Planificación**: `plan_path_only()` calcula el camino con visgraph sin ejecutarlo.
7. **Navegación**: el robot recorre el camino con detección de colisiones y replanificación automática.
8. **Verificación YOLOE**: en 3 stages (standoff, 360° scan, alternativa).
9. **Retry**: si falla, siguiente candidato (hasta 8). Si se agotan, siguiente categoría.

4.2. Flujo completo del heatmap

Paso 1 — Scores CLIP por vóxel

Cada vóxel del mapa 3D tiene un vector de scores CLIP de dimensión $|\mathcal{C}|$. Para una query dada, se extrae el score $s_i \in [0, 1]$ de cada vóxel i para la categoría objetivo. Solo se retienen vóxeles con argmax en esa categoría y $s_i > 0{,}3$.

Paso 2 — Proyección 2D (max-pool por columna)

Los scores 3D se proyectan sobre el plano del suelo tomando el máximo por celda (r, c). Resultado: heatmap flotante $gs \times gs$ (por defecto $gs = 960$).

Paso 3 — Difusión por distancia

Difusión basada en `distance_transform_edt`: caída lineal $\times 0{,}3$ por celda. Celdas con score $< 0{,}15$ se anulan (franja $\sim$3 celdas = 15 cm).

Paso 4 — Suavizado Gaussiano

Gaussian blur 5×5, $\sigma = 1{,}0$ para suprimir ruido de alta frecuencia.

Paso 5 — Umbral relativo y componentes conectados

Binarización con $t = 0{,}5 \times$ máx(heatmap suavizado). Componentes conexos (8-conectividad). Para cada componente: área, score máximo, score medio, bounding box, centroide.

Paso 6 — Scoring interno y filtro relativo

$$score(r) = mean_val(r) \cdot \log(1 + area(r))$$

Se descartan componentes con $score(r) < 0{,}2 \cdot \text{máx}_j\, score(j)$.

Paso 7 — Fórmula de calidad

$$quality(r) = 0{,}3 \cdot \widehat{area}(r) + 0{,}4 \cdot \overline{s}(r) + 0{,}3 \cdot \delta(r)$$

donde $\widehat{area}$ es el área normalizada, $\overline{s}$ el score CLIP medio y δ la densidad (celdas activas / bounding box).

Paso 8 — Selección del goal

`select_safe_goal_from_path()` recorre el camino planificado y elige el último waypoint con score no nulo, garantizando visibilidad directa al objeto.

4.3. Proveedor de habitaciones HSSD

La clase `SemanticSceneRoomProvider` (en `room_provider.py`) lee el fichero `semantics/scenes/<id>.semantic` de cada escena HSSD y expone:

- `build(config_path, gs, hab_to_grid)`: lee `region_annotations`, convierte cada polígono de Habitat world a coordenadas del grid VLMap mediante `hab_to_grid(x, z)`, y rasteriza con test punto-en-polígono. Resultado: `_region_grid[gs, gs]` (int32, 0 = sin etiquetar).
- `get_room_at_cell(row, col)` → nombre de la habitación o `None`.
- `get_room_centroid(room_name)` → (row, col) del centro de la habitación.
- `list_rooms()` → lista de nombres de todas las habitaciones.

Cada región almacena: `id`, `name`, `category`, `centroid [r, c]`, `poly_xz`, `floor_height`.

4.4. Archivos clave del proyecto

Archivo	Rol
`application/interactive_object_nav.py`	Pipeline de navegación principal
`vlmaps/robot/habitat_lang_robot.py`	Robot + visgraph + `plan_path_only`
`vlmaps/utils/llm_utils.py`	Mapper LLM: objeto → categorías + habitaciones
`vlmaps/utils/yoloe_utils.py`	YoloeSession: worker persistente, sync/async
`vlmaps/utils/_yoloe_persistent_worker.py`	Subprocess YOLOE aislado
`vlmaps/utils/room_provider.py`	SemanticSceneRoomProvider (HSSD nativo)
`vlmaps/utils/room_map_utils.py`	Room segmentation + `find_room_goal()`
`vlmaps/utils/navigation_utils.py`	`plan_to_pos_v2`: visgraph + BFS
`config/object_goal_navigation_cfg.yaml`	Config Hydra principal
`docker/menu.sh`	Menú interactivo Docker

5. Próxima fase: búsqueda con razonamiento por habitaciones

5.1. Visión general

El sistema actual trata la escena como un mapa plano: genera un heatmap global, elige el candidato más cercano o de mayor calidad, y navega hacia él sin considerar la estructura de la vivienda. Esto implica que el robot puede cruzar toda la casa para ir a un candidato ruidoso cuando la habitación correcta está al lado.

La siguiente fase introduce una **capa de razonamiento por habitaciones** que separa la decisión de búsqueda en dos niveles:

- **Capa alta**: ¿en qué habitación o zona tiene más sentido buscar ahora?
- **Capa baja**: dentro de esa habitación, ¿qué candidato concreto priorizo?

Esto permite comportamientos como: buscar `sink` primero en `bathroom`; si `bathroom` falla, probar `kitchen`; dentro de `kitchen`, priorizar `counter` o `cabinet` según la query.

La arquitectura se implementa en 8 fases incrementales (A–H). Cada fase añade una capacidad concreta sin romper lo anterior.

5.2. Fase A — Capa de habitaciones sobre HSSD — COMPLETADA

Objetivo: dejar de tratar la escena como un mapa plano.

Implementación realizada:

- `SemanticSceneRoomProvider.build()` recibe una función `hab_to_grid(x, z)` que transforma coordenadas Habitat world a índices del grid VLMap usando `init_hab_tf` y `base_transform`. Los polígonos de habitación se rasteriza directamente en espacio grid.
- Cada **celda del grid** tiene room_id via `get_room_at_cell(row, col)`.
- Cada **centroide de componente** del heatmap se anota con su habitación. El sistema imprime `Current room` y `Candidates by room` en cada query.
- Si la instrucción es un **nombre de habitación** (e.g., *"go to the kitchen"*), el robot navega directamente al centroide de la habitación sin calcular heatmaps ni ejecutar verificación YOLOE. Al llegar, se detiene.

Nota: la verificación visual de llegada a una habitación (confirmar que el robot está en la cocina mirando objetos típicos) no es necesaria en simulación, donde las coordenadas son exactas. Para

un robot real con localización imperfecta, esta verificación podría reactivarse como capa adicional de seguridad.

Pendiente para fases posteriores: el parsing LLM actual separa instrucciones compuestas como *"find the sofa in the living room"* en dos targets independientes (`['sofa', 'living_room']`), ejecutándolos secuencialmente. El sistema necesita un parsing semántico por niveles que interprete la instrucción completa: habitación objetivo + objeto dentro de ella. Esto es requisito para instrucciones complejas tipo *"ve a por la botella de agua que está encima de la mesa de la cocina"*.

5.3. Fase B — Representación de estado por habitación — COMPLETADA

Implementada en `vlmaps/utils/search_state.py`.

Estructura real implementada:

```
@dataclass
class RoomState:
    name: str
    centroid: Tuple[float, float]   # (row, col) en grid VLMap
    total_cells: int                # celdas de región en region_grid
    free_cells: int                 # celdas navegables (intersec. obstacles_map)
    times_visited: int
    objects_seen: List[str]         # confirmados por YOLOE en esta room
    candidates_tried: int
    candidates_confirmed: int
    target_relevance: float         # prior Fase C (0-1, normalizado)
    target_found_here: bool
    # derived:
    explored_ratio = free_cells / total_cells
```

`SearchState` agrega un `RoomState` por habitación y registra: `visit_history` (secuencia de rooms visitadas), `current_room`, `found`. Se construye una instancia por target al inicio de cada instrucción. Métodos: `update_current_room()`, `record_candidate()`, `record_object_seen()`, `mark_found()`, `compute_priors()`.

5.4. Fase C — Priors objeto → habitación — COMPLETADA

Implementada en `vlmaps/utils/room_priors.py`. Integrada en `SearchState.compute_priors()` y en el bucle principal de `interactive_object_nav.py`.

Fórmula de fusión:

$$prior_final(r) = 0{,}5 \cdot prior_llm(r) + 0{,}3 \cdot prior_manual(r) + 0{,}2 \cdot evidence(r)$$

Los scores se recortan a $[0, 1]$ y se normalizan para que sumen 1.

Tres fuentes implementadas:

1. **LLM** (`_query_llm_room_prior`): GPT-4o-mini recibe la query y la lista de habitaciones conocidas de la escena y devuelve un JSON `{room: score}`. Robusto ante fallos de red (devuelve dict vacío).
2. **Tabla manual** (`_MANUAL_TABLE`): ∼35 entradas clave (e.g. `bed`→`bedroom:1.0`, `sink`→`bathroom:0.9`, `kitchen:0.7`). Matching por subcadena entre fragmento de tabla y nombre real de habitación en escena.

3. **Evidencia escena** (`_evidence_prior`): tabla de co-ocurrencia (`_CO_OCCURRENCE`) — si ya se vieron `toilet` o `bathtub` en una habitación, su score de `bathroom` se refuerza. Usa `RoomState.objects_seen` (actualizado en tiempo real).

Integración en el pipeline: al inicio de cada instrucción de búsqueda se imprime:

```
Room priors for 'sink': bathroom=0.62, kitchen=0.28, living room=0.07, ...
```

El resultado se almacena en `RoomState.target_relevance` de cada habitación, listo para ser consumido por el selector de Fase D.

5.5. Fase D — Selector de habitación

Objetivo: elegir la mejor habitación para buscar en cada momento.

En vez de mandar al robot directamente al mejor candidato global, se calcula una puntuación por habitación:

$$room_score(r) = w_1 \cdot prior(r) + w_2 \cdot evidence(r) + w_3 \cdot unexplored(r) - w_4 \cdot cost(r) - w_5 \cdot penalty(r)$$

Donde:

- $prior(r)$: relevancia de la habitación para la query, según Fase C.
- $evidence(r)$: suma de scores de calidad VLMap de los candidatos dentro de la habitación.
- $unexplored(r) = 1 - explored_ratio(r)$: preferir habitaciones no agotadas.
- $cost(r)$: distancia BFS desde la posición actual del robot al centroide de la habitación, normalizada.
- $penalty(r)$: penalización acumulada por intentos fallidos previos en esa habitación.

Los pesos $w_1 \dots w_5$ se ajustan empíricamente. Valores iniciales sugeridos: $w_1 = 0{,}3$, $w_2 = 0{,}25$, $w_3 = 0{,}2$, $w_4 = 0{,}15$, $w_5 = 0{,}1$.

Esfuerzo estimado: 1–2 sesiones.

5.6. Fase E — Selección de candidatos intra-habitación

Objetivo: una vez elegida la habitación, reutilizar el pipeline existente dentro de ella.

1. Filtrar `kept_components` del heatmap a aquellos cuyo centroide cae dentro de la habitación seleccionada (via `get_room_at_cell()`).
2. Ordenar los candidatos filtrados por `quality` (fórmula ya implementada).
3. Navegar al mejor candidato con la pipeline M2 existente (visgraph + colisiones + YOLOE).
4. Si no hay candidatos sólidos en la habitación, explorar el frontier más cercano dentro de ella.
5. Si la query es de ubicación conocida (e.g., "ve a la cocina"), navegar directamente al centroide de la habitación.

Esto **no descarta la pipeline actual**: la reutiliza con un filtro espacial por habitación.

Esfuerzo estimado: 1 sesión.

5.7. Fase F — Acciones de alto nivel

Objetivo: definir un vocabulario de acciones que abstraiga la navegación de bajo nivel.

Acción	Descripción
`go_to_room(room)`	Navegar al centroide de la habitación
`explore_room(room)`	Cobertura sistemática de celdas no visitadas dentro de la habitación
`inspect_candidate(room, cand)`	Navegar a candidato específico + verificar con YOLOE
`verify_target`	Scan 360° + visual servoing en posición actual
`done`	Objetivo encontrado, fin de la búsqueda

Internamente, `go_to_room` e `inspect_candidate` usan la navegación visgraph existente. `explore_room` es nuevo: genera waypoints que cubran las celdas navegables no visitadas de la habitación.

Esfuerzo estimado: 1–2 sesiones.

5.8. Fase G — Papel del LLM en la búsqueda

Objetivo: el LLM decide estrategia, no trayectoria.

El LLM recibe un **estado estructurado compacto** y decide qué habitación visitar y qué acción ejecutar. No recibe contexto bruto enorme: recibe un resumen estructurado.

Ejemplo de prompt interno:

```
Task: find sink

Current room: hallway

Known rooms:
  - bathroom: objects [mirror, toilet, shower], explored 80%,
              1 candidate tried (vanity-zone: YOLOE no confirmation)
  - kitchen:  objects [counter, cabinet, table], explored 60%,
              0 candidates tried
  - living_room: objects [sofa, table, shelf], explored 90%

Unexplored zones:
  - bathroom: near rear corner
  - kitchen: near counter area

Which room should be searched next, and which category or zone
inside it should be prioritized?
```

El LLM devuelve la habitación y zona sugeridas. La puntuación por habitación (Fase D) se usa como *fallback* numérico si el LLM no da respuesta clara, o como validación cruzada.

Historial compacto: solo las últimas 5 acciones, habitaciones exploradas, candidatos probados y motivo de fallo (no detectado / heatmap débil / navegación fallida). Esto evita inflar el prompt.

Esfuerzo estimado: 2 sesiones.

5.9. Fase H — Métricas de evaluación

Objetivo: medir si el razonamiento por habitaciones mejora la búsqueda.

Métricas estándar de navegación semántica:

- **SR** (Success Rate): fracción de episodios donde YOLOE confirma la detección.
- **SPL** (Success weighted by Path Length): SR ponderado por la eficiencia del camino.

- **Wrong Visits**: candidatos visitados sin detección positiva antes del éxito.
- **Time to Find**: pasos de simulación hasta la confirmación.

Métricas nuevas específicas del razonamiento por habitaciones:

- **CFR** (Correct First Room): ¿fue la primera habitación visitada la que contenía el objetivo?
- **CT2R** (Correct Top-2 Rooms): ¿estaba la habitación correcta entre las 2 primeras visitadas?
- **Rooms Before Success**: número de habitaciones visitadas antes de encontrar el objeto.
- **Inspections Before Success**: número de candidatos inspeccionados antes del éxito.

Estas métricas permiten demostrar si el razonamiento por habitaciones aporta valor real frente a una búsqueda plana.

5.10. Orden de implementación

Fase	Descripción	Depende de	Esfuerzo
A	Capa de habitaciones sobre HSSD	Infraestructura existente	Hecho
B	RoomState + SearchState	Fase A	Hecho
C	Priors objeto → habitación	Mapper LLM existente	Hecho
D	Selector de habitación (scoring)	Fases B + C	1–2 sesiones
E	Candidatos intra-habitación	Fase D + pipeline M2	1 sesión
F	Acciones de alto nivel	Fase E	1–2 sesiones
G	LLM estratégico con estado	Fases B + C + F	2 sesiones
H	Métricas de evaluación	Fase F	1 sesión
	Total estimado		**9–12 sesiones**

Las fases A–C son preparatorias y pueden implementarse rápidamente sobre la infraestructura existente. Las fases D–E son el núcleo del cambio de comportamiento. Las fases F–G añaden la inteligencia de alto nivel. La fase H cierra el ciclo con evaluación.

6. Fase posterior: nuevos objetos y evaluación

Una vez consolidado el razonamiento por habitaciones, el proyecto pasa a una fase de evaluación y ampliación:

6.1. Baselines M1 y M3

- **M1 — Navegación aleatoria**: el robot elige celdas navegables al azar como goals. Sirve como cota inferior de rendimiento.
- **M3 — Ranking por calidad**: candidatos ordenados por `quality` (fórmula ya implementada) sin razonamiento por habitaciones. Mide si la calidad semántica aporta sobre la proximidad pura (M2).

Ambos baselines se implementan como variantes del pipeline existente y se comparan con el sistema room-aware de la Sección 5.

6.2. Evaluación cuantitativa

Script automático que ejecuta N trials con objetos conocidos y genera:

- CSV con resultados por episodio (SR, SPL, Wrong Visits, CFR, CT2R, etc.)

- Gráficas comparativas: M1 vs M2 vs M3 vs room-aware
- Tablas para la memoria del TFG

Para HSSD: usar las anotaciones nativas del `semantic_config.json` para generar trials con ground truth automatizado.

6.3. Objetos adicionales en simulación

Usar `place_objects.py` (ya implementado) para insertar objetos de interés:

- `bottle`, `backpack`, `ball`, `laptop`
- Colocados en posiciones conocidas para evaluación con ground truth
- Visibles para YOLOE, no presentes en el VLMap pre-construido

7. Extensión futura: ROS 1 Noetic + Toyota HSR

El proyecto se desarrolla y valida primero en simulación, con HSSD como entorno principal y con foco en la capacidad de razonamiento por habitaciones y búsqueda semántica de objetos. Una vez consolidada la capa de decisión de alto nivel y evaluado el sistema sobre tareas representativas, se contempla como línea de extensión la migración a ROS 1 Noetic y su integración con el robot Toyota HSR.

La arquitectura de razonamiento por habitaciones (Sección 5) está diseñada para ser independiente de la plataforma de ejecución: las capas de decisión (selector de habitación, LLM estratégico, estado de búsqueda) no dependen de Habitat-Sim. La migración sustituiría únicamente las capas de percepción, navegación y ejecución por sus equivalentes en robot real:

- `vlmap_server` (nodo ROS): carga el VLMap y expone un servicio `/vlmap/query` para consultas semánticas.
- `vlmap_nav` (nodo ROS): recibe instrucciones en lenguaje natural, ejecuta el razonamiento por habitaciones y envía goals a `move_base`.
- Visualización en RViz: heatmap como `OccupancyGrid`, camino como `nav_msgs/Path`, estado de habitaciones como markers.
- Cámara RGBD del HSR como entrada para YOLOE (sustituyendo la cámara simulada de Habitat-Sim).

Esta extensión no forma parte del roadmap inmediato. Su viabilidad depende de que la capa de razonamiento funcione correctamente en simulación y de la disponibilidad del robot HSR en el laboratorio V4R del TU Wien.

8. Diario de desarrollo

31 de marzo de 2026 — Correcciones del pipeline HSSD

1. **Mapa de obstáculos invertido**: modelo de dos bandas (suelo $h < 0{,}15\,\mathrm{m}$ / obstáculo $0{,}2 \leq h < 1{,}5\,\mathrm{m}$). Una celda navegable = 1 si tiene suelo Y no tiene obstáculo.
2. **Heatmap ruidoso**: argmax binario reemplazado por score CLIP continuo con umbral 0.3 + max-pooling + difusión solo desde celdas con score alto.
3. **Robot atraviesa paredes en HSSD**: `sim.recompute_navmesh()` en `_setup_sim()` cuando `dataset_type == "hssd"`.

4. **Integración YOLOE**: `yoloe_utils.py` + `_yoloe_worker.py`, superposición sobre ventana 1st person.
5. **Camino visible**: `path_cells` pasado a `show_map()`, color rojo.
6. **Cámara primera persona**: inclinación $-\pi/8$ rad ($\approx$ 22,5).

2 de abril de 2026 (primera iteración)

1. **Postproceso de heatmaps**: suavizado Gaussiano $\rightarrow$ umbral relativo $\rightarrow$ metadatos por componente (área, max, media, bbox, centroide).
2. **Exploración 360°**: pasos de 20°, detención al detectar.
3. **Ruta alternativa**: componente con mayor score a >20 celdas.
4. **Movimiento fluido**: sin `set_agent_state()` visible.

2 de abril de 2026 (segunda iteración)

1. **Worker YOLOE persistente**: modelo cargado una sola vez; protocolo stdin/stdout; inferencias $\sim$100–500 ms.
2. **YoloeSession**: modos síncrono y asíncrono (hilo bg).
3. **Scan 360° fluido**: pasos nativos de 5°, YOLOE en hilo bg.
4. **Cámara** -30: `habitat_utils.py`.
5. **Standoff 0,25 m**: acercamiento máximo al objeto.

13 de abril de 2026

1. **Bug protocolo stdout/stderr YOLOE**: `_ensure_mobileclip()` imprimía en stdout interceptando el `"READY"`; corregido a `sys.stderr`. `start()` ahora lee hasta recibir `"READY"`.
2. **mobileclip_blt.ts**: búsqueda en múltiples rutas candidatas; eliminación automática de versión corrupta (<100 MB).
3. **Captura stderr del worker**: crashes visibles en consola.
4. **Dirección de cara definitiva**: $\arctan 2(-\Delta y, -\Delta x)$, signo consistente con el planificador.
5. **Sin teletransporte**: `plan_path_only()` + camino 800 ms antes de ejecutar.
6. **Hydra corregido**: `scene_dataset_config_file` sin prefijo `+` en ejecución y documentación de `menu.sh`.
7. **Repositorio GitHub limpio**: `mobileclip_blt.ts` (571 MB) eliminado del historial con `git-filter-repo`; push operativo.
8. **Bugs corregidos**: IndexError boundary (clamp en `plan_path_only()`) + umbral YOLOE subido a 0,40.
9. **Fórmula de calidad**: implementada en `postprocess_heatmap()`; componentes ordenados por quality.

14 de abril de 2026

1. **Reestructuración del documento de proyecto**: se redefine la finalidad del proyecto con foco en razonamiento por habitaciones.

2. **HSSD confirmado como dataset principal**: MP3D queda como soporte secundario; LabelMe diferido.
3. **Arquitectura room-aware diseñada**: 8 fases (A–H) con representación de estado por habitación, scoring de selección de room, acciones de alto nivel y rol estratégico del LLM.
4. **Roadmap corregido**: la escalera M1–M5 se reemplaza por la arquitectura room-aware como prioridad central. Baselines y evaluación pasan a fase posterior.
5. **Fase A implementada**: `SemanticSceneRoomProvider` construido con transformación completa Habitat→grid (`hab_to_grid`). Polígonos rasterizados en espacio grid. Cada componente del heatmap anotado con su habitación. Navegación a habitación por nombre funcional (centroide directo, sin heatmap ni YOLOE). Verificación visual de llegada a habitación omitida en simulación (coordenadas exactas); se contempla reactivarla para robot real con localización imperfecta.
6. **Parsing semántico por niveles**: identificado como trabajo futuro necesario. El parsing actual separa instrucciones compuestas en targets independientes; se requiere un parser que interprete habitación objetivo + objeto dentro de ella como una sola instrucción estructurada.

15 de abril de 2026

1. **Fase B completada**: `RoomState` y `SearchState` implementados en `vlmaps/utils/search_state.py`. Cada `RoomState` registra `explored_ratio`, `objects_seen`, `candidates_tried/confirmed`, `times_visited`, `target_relevance` y `target_found_here`. Se construye una instancia por target al inicio de cada instrucción y se actualiza incrementalmente durante la navegación y la verificación YOLOE. Al final de cada instrucción se imprime un resumen completo del estado de búsqueda.
2. **Fase C completada**: módulo `vlmaps/utils/room_priors.py`. Implementa `compute_room_priors(query known_rooms, objects_seen_by_room)` fusionando tres fuentes con pesos $a = 0{,}5$, $b = 0{,}3$, $c = 0{,}2$: (a) LLM GPT-4o-mini con llamada estructurada al inicio de cada búsqueda de objeto; (b) tabla manual de ∼35 objetos con scores por tipo de habitación; (c) evidencia de co-ocurrencia basada en `objects_seen` por habitación. El resultado normalizado se almacena en `RoomState.target_relevance` y se muestra como traza de depuración antes de cada consulta.
3. **Navegación solo a candidatos del heatmap**: eliminado el uso de `get_standoff_pos()` (que usaba `labeled_map_cropped`). Ahora el robot navega directamente al centroide del mejor componente de `kept_components` (filtrado real por argmax + score $> 0{,}3$). Esto evita que el robot viaje a zonas con señal VLMap pero sin activación real en el heatmap.
4. **Filtrado de objetos navegables**: la lista de objetos mostrada antes de cada consulta usa el mismo filtro que `compute_heatmap` (vóxeles con argmax ganador y score $> 0{,}3$, mínimo 10 vóxeles), garantizando coherencia entre lo que se muestra y lo que el sistema puede encontrar.
5. **Correcciones de navegación**: (a) `n_actions == 0` tratado como "ya en el destino" en vez de "sin camino", corrigiendo el caso post-llegada a habitación; (b) eliminado el bloque de reintento por paso estrecho (*narrow-passage retry*) que generaba más falsos positivos que beneficios; (c) umbral YOLOE subido a 0,3 y dilatación de obstáculos a 3 iteraciones (≈15 cm por lado).
6. **Tipos canónicos de habitación (*room_type* vs *room_instance*)**: HSSD codifica instancias repetidas de un mismo tipo de habitación con sufijos numéricos (`bathroom.001`, `bathroom.002`, etc.). Se añade `canonical_room_type()` en `room_priors.py` que elimina

el sufijo `.NNN` mediante regex (`re.sub(r'\.\d+$', '', s)`), y `compatible_room_types()` que devuelve el conjunto de tipos canónicos con prior $> 0{,}05$. Estas funciones permiten razonar sobre familias de habitación independientemente del índice de instancia.

7. **Comportamiento *local-first* al navegar (*room gate*)**: nueva función `select_best_candidate(kept_c current_room, query_priors, tried_centroids)` en `interactive_object_nav.py`. Si el robot ya se encuentra dentro de una instancia de habitación cuyo tipo canónico es compatible con el objeto buscado (e.g. robot en `bathroom.001`, objeto `sink`), el sistema prioriza los candidatos del heatmap situados en esa misma instancia, aplicando un bonus de $+0{,}35$ a su calidad efectiva. El cambio de habitación sólo se produce si (a) no quedan candidatos locales sin explorar, o (b) la calidad externa supera la local en más de un 40 % tras el bonus. El conjunto `_tried_centroids` en `SearchState` registra los centroides ya visitados para evitar ciclos.

8. **Navegación con margen de seguridad aumentado**: la dilatación de obstáculos en `habitat_lang_robot.py` se eleva a 5 iteraciones ($\approx$25 cm por lado). La función `select_safe_goal_from_p` aumenta el radio mínimo de *clearance* de 3 a 5 celdas y adopta estrategia de máxima holgura: reúne *todos* los puntos del camino que cumplen el radio y selecciona el de mayor distancia al obstáculo más cercano (en vez del primero válido caminando hacia atrás). Si la pasada estricta no devuelve candidatos, una segunda pasada suaviza el umbral y vuelve a seleccionar por máxima holgura.

9. **Métricas de seguridad por camino**: nueva función `_compute_path_safety(path_cells, obs_map)` basada en `distance_transform_edt`. Registra longitud, holgura mínima y media, penalización acumulada por celdas con clearance < 8 y coste seguro $=$ longitud $+$ $0{,}5 \cdot$ penalización. Los valores se imprimen en trazas `[safety]` tras cada planificación, facilitando el diagnóstico de rutas peligrosas y la futura migración a ROS/HSR.

10. **Corrección Bug 1 — comandos de habitación con validación real de llegada**: los comandos de tipo `kitchen`, `dining room`, etc. fallaban porque el sistema navegaba al centroide geométrico del polígono de la habitación (a menudo sobre un obstáculo), el planificador lo desplazaba a un nodo visgraph cercano (frecuentemente en otra habitación) y después declaraba éxito sin comprobar dónde estaba realmente el robot. Se implementa `find_reachable_room_goal()`, que enumera todas las celdas navegables del interior de la habitación objetivo, las ordena por holgura respecto a obstáculos y devuelve los top-K candidatos seguros. Tras la ejecución del camino se comprueba la habitación real con `get_room_at_cell()` y se valida mediante `_room_instance_matches()`, que acepta tanto coincidencia exacta como de tipo canónico (`kitchen.001` satisface el comando `kitchen`). Si la primera meta falla, se reintenta con los candidatos alternativos antes de declarar fracaso. Solo se imprime "Arrived at room X" cuando la comprobación pasa.

11. **Corrección Bug 2 — evidencia del heatmap domina en consultas directas**: para consultas de objeto con señal real en el heatmap (p.ej. `sofa`), los priors de habitación dependían excesivamente de los priors lingüísticos (LLM + tabla manual), ignorando que los componentes del heatmap ya indicaban en qué habitación está el objeto. Además, nombres de habitación específicos de la escena como `tv` no coincidían con los fragmentos estándar (`living`) de la tabla manual. Se añade la tabla `_SEMANTIC_ALIASES` que mapea nombres atípicos a familias semánticas (`tv` $\rightarrow$ `living`, `laundryroom` $\rightarrow$ `laundry`, etc.) y se aplica en `_manual_prior()` y `_evidence_prior()` mediante `_normalize_room_for_priors()`. Se añade `compute_heatmap_room_evidence()` que agrega la calidad de componentes por instancia de habitación y normaliza a $[0, 1]$. Se extiende `compute_room_priors()` con los parámetros opcionales `heatmap_evidence` y `query_type`: las consultas directas usan pesos $0{,}65 \cdot \text{heatmap} + 0{,}20 \cdot \text{manual} + 0{,}15 \cdot \text{LLM}$, mientras que las indirectas mantienen $0{,}45 \cdot \text{LLM} + 0{,}30 \cdot \text{manual} + 0{,}25 \cdot \text{evidencia escena}$. En el bucle de navegación, tras anotar

los componentes con su habitación, se recalculan los priors con evidencia real antes de seleccionar el candidato, de modo que `sofa` con tres componentes en `tv` produce un prior $> 0{,}86$ para `tv`.

12. **Corrección Bug 3 — puerta de seguridad bloquea ejecución de rutas inseguras**: las métricas de seguridad `[safety]` existían pero eran puramente informativas: el sistema ejecutaba igualmente rutas con `goal_cl=0.0` o `min_cl=0.0`. Se añaden umbrales duros configurables `_MIN_GOAL_CLEARANCE=3,0` celdas y `_MIN_PATH_CLEARANCE=1,0` celdas. Antes de ejecutar cualquier camino a un objeto, se evalúan dichos umbrales: si el goal o la ruta no los superan, el centroide se marca como intentado en `_tried_centroids` y se prueban hasta tres candidatos alternativos. Si ningún candidato supera los umbrales, la consulta se omite con mensaje explícito. La seguridad ya no es anotación, sino control de flujo.

13. **Corrección Problema 1 — metas de aproximación seguras alrededor del centroide de componente**: el pipeline anterior planificaba hasta el centroide del heatmap (a menudo situado sobre un obstáculo), ejecutaba el camino completo y luego recorría el camino hacia atrás buscando un buen punto de observación. Si el centroide caía dentro de un obstáculo, el planificador visgraph producía rutas erróneas. Se añade `find_safe_candidate_approach_goa` `safe_obs_map, robot_pos)` que muestrea posiciones de aproximación en un anillo de 8–25 celdas alrededor del centroide, filtra por navegabilidad en `_safe_obs_map` y holgura mínima ≥ 3 celdas, y ordena por holgura descendente y proximidad al robot. La meta de navegación se elige directamente del mejor candidato del anillo; el camino hacia atrás solo se activa como respaldo si el anillo queda vacío.

14. **Corrección Problema 2 — planificador global atravesaba paredes**: causa raíz doble. (a) La dilatación de obstáculos con 5 iteraciones (bilateral $\approx$ 10 celdas) cerraba los pasillos estrechos de HSSD ($\sim$10 celdas de ancho) desconectando habitaciones en el visgraph. Corregido volviendo a 3 iteraciones ($\approx$ 6 celdas). (b) El método de recuperación de goal en espacio de obstáculo usaba `G.closest_point()` (holgura $= 0$) con `get_nearby_position()` como fallback, que devuelve `None` si las cuatro diagonales están bloqueadas; `vg.Point(None[0])` genera coordenadas inválidas $(-1, 96)$ y `G.shortest_path` produce una línea recta a través de paredes. Ambas funciones se reemplazan por `_snap_to_nearest_free(p` `obstacles, min_clearance)` en `navigation_utils.py`, que usa `distance_transform_edt` para encontrar la celda libre más cercana con holgura suficiente: el resultado está siempre garantizado dentro del espacio navegable del mapa del visgraph. Se añaden también registros de diagnóstico `[planner] Start navigable: ... Goal navigable: ...` y almacenamiento del mapa dilatado completo como `robot._safe_obs_map` para que el resto del sistema use la misma referencia que el visgraph. Las funciones `find_reachable_room_goal()` y `find_safe_candidate_approach_goals()` se actualizan para usar `robot._safe_obs_map` en lugar del mapa de navmesh crudo, eliminando la discordancia entre validación de metas y planificador.

15. **Corrección Problema 3 — escudo de colisiones local en ejecución**: `execute_nav_replay()` acepta ahora el parámetro opcional `dist_map` (transformada de distancias del mapa seguro dilatado, precalculada una vez antes de cada ejecución). Antes de cada acción `move_forward`, el escudo predice la siguiente celda de cuadrícula a partir de la posición actual `curr_pos_on_map` y el ángulo `curr_ang_deg_on_map` del robot. Si la holgura en esa celda es inferior a `STOP_CL` $= 2{,}0$ celdas, la ejecución se interrumpe inmediatamente y se activa la recuperación; si la holgura está entre `STOP_CL` y `SLOW_CL` $= 5{,}0$ celdas se emite un aviso sin detener el movimiento. El escudo opera sobre el mismo mapa dilatado que el visgraph, de modo que «celda segura» tiene idéntico significado en planificación y en ejecución.

16. **Navegación one-shot — eliminados reintentos y recuperación**: el escudo de colisiones usaba el mapa dilatado (`_safe_obs_map`) para calcular la transformada de dis-

tancias, lo que hacía que el centro de un pasillo de ~ 10 celdas tuviera clearance ≈ 2 en el mapa dilatado y disparara paradas falsas al cruzar puertas. El escudo pasa a usar `robot.map.obstacles_map` (mapa crudo sin dilatar), que refleja el espacio físico real; los umbrales se ajustan a `STOP_CL` = 1,0 y `SLOW_CL` = 2,5 celdas. Adicionalmente, se elimina la llamada a `nav_recovery_and_replan()` tras un fallo de ejecución y el bucle de reintentos con metas alternativas en comandos de habitación. La navegación es ahora estrictamente one-shot: se planifica una vez, se ejecuta una vez y se reporta el estado real de la habitación; si la ejecución se interrumpe, el robot continúa desde la posición actual sin ruta nueva.

16 de abril de 2026

1. **Separación explícita entre inflación de planificación y escudo de ejecución**: el planificador global utiliza el mapa dilatado (`_safe_obs_map`, 3 iteraciones de dilatación binaria, ≈15 cm por lado) para trazar rutas con suficiente margen. El escudo de ejecución opera exclusivamente sobre la transformada de distancias del mapa crudo (`robot.map.obstacles_map`), sin dilatar. Esto garantiza que los pasillos estrechos de HSSD (∼10 celdas de ancho, clearance real ≈5 celdas) no sean bloqueados falsamente por el escudo. La constante `_PLAN_DILATION_ITERS = 3` se declara de forma explícita en `habitat_lang_robot.py` y se imprime al inicio de la sesión junto con el mensaje: «*global planner only; execution shield uses raw map*».

2. **Modo de travesía de puertas (*doorway mode*)**: antes de cada `move_forward`, el escudo mide las clearances perpendiculares a la derecha y a la izquierda del robot en la pose actual (`_side_l`, `_side_r`). Si ambas caen por debajo de `DOORWAY_SIDE_TH` = 5,0 celdas y la clearance frontal supera `DOORWAY_FRONT_MIN` = 0,5 celdas, se activa el modo de paso estrecho. En ese modo el umbral de parada dura se relaja a `STOP_CL_DOOR` = 0,5 celdas (frente a 1,0 en espacio abierto) y cada paso se registra como «*cautious doorway traversal*». La transición ON/OFF se imprime explícitamente. El planificador puede enrutar libremente por puertas válidas; el escudo deja cruzarlas sin falsas paradas.

3. **Escudo sensible a la huella completa del robot**: el chequeo anterior solo evaluaba un punto (celda frontal prevista). Ahora se evalúan tres puntos de la huella en la pose predicha: *front-center*, *front-left* y *front-right* (desplazados lateralmente `ROBOT_HW` = 2 celdas respecto al eje de avance). Se registra por separado `front clearance`, `left clearance`, `right clearance` y `footprint min clearance`. En espacio abierto, si algún lateral cae por debajo de `SIDE_STOP_CL` = 1,5 celdas, el paso se bloquea. En modo puerta, el criterio de bloqueo es solo el mínimo de la huella frente a `DOORWAY_FRONT_MIN`.

4. **Puerta de entrada a habitación para confirmación YOLOE (*room-entry gate*)**: hasta ahora el sistema podía declarar éxito inmediatamente si YOLOE detectaba el objeto desde la puerta antes de haber cruzado el umbral. Se añade una comprobación en los stages 0 y 1 de YOLOE: si el robot está en una habitación distinta de la que contiene el componente del heatmap objetivo (`best_comp["room"]`) y la distancia al centroide del candidato supera 20 celdas (≈1 m), la detección se marca como *tentativa* (`_yoloe_confirmed = False`) y la búsqueda continúa con el stage siguiente. Solo se acepta como éxito definitivo si el robot ha entrado en la habitación correcta o se encuentra a menos de 20 celdas del candidato. Se registra en consola «*Object visible but room not yet entered → tentative only*» o «*Room entry confirmed*» según el caso.

5. **Centrado de trayectorias en pasos estrechos mediante eje medial local**: el visgraph seguía generando caminos geométricamente válidos pero sesgados hacia uno de los lados del estrechamiento, lo que hacía que el escudo de ejecución bloquease pasos por clearance lateral insuficiente aun estando en modo de paso estrecho. El primer intento de corrección solo recentraba vértices ya existentes y no bastaba cuando el estrechamiento caía en mitad de un

segmento largo del visgraph. Se añade un postprocesado `center_path_on_medial_axis()` en `vlmaps/utils/navigation_utils.py`. Tras obtener la ruta del visgraph, se rasteriza cada segmento y, si detecta una franja con clearance $< 4{,}0$ celdas, inserta un waypoint auxiliar en la zona más estrecha. Ese waypoint, junto con los intermedios originales, se desplaza hacia la celda de mayor holgura dentro de un disco local de radio 6, usando `distance_transform_edt` del mapa *sin* dilatar. El candidato solo se acepta si mantiene línea de visión libre con los waypoints vecino anterior y siguiente sobre el mapa de planificación dilatado, evitando atajos que corten esquinas o crucen paredes. Después se aplica un suavizado conservador de 3 puntos y se conserva la meta final sin desplazar. El resultado es que la ruta cruza por el centro del paso incluso cuando el cuello de botella no coincide con un vértice del visgraph, y el escudo vuelve a comportarse como mecanismo de seguridad, no como corrector sistemático de trayectorias.

6. **Modo de paso estrecho activado por geometría local, no solo por simetría lateral**: en ejecución real aparecía un caso límite al entrar oblicuamente en un marco o cuello de botella: un lateral del footprint caía por debajo de 1,5 celdas mientras el otro seguía aún en espacio relativamente abierto, de modo que el detector antiguo no activaba el modo de paso estrecho y el escudo bloqueaba el avance con la regla de espacio abierto. Se refuerza la detección en `execute_nav_replay()` con tres señales: (1) estrechamiento en la pose actual, (2) estrechamiento en la huella de la pose siguiente y (3) entrada asimétrica, cuando un lateral ya es crítico pero la clearance frontal sigue siendo cruzable. Además se añade histéresis de salida para mantener el modo activo mientras el robot permanezca dentro del cuello de botella. El modo pasa a modelar *pasos estrechos cruzables* en general, no solo puertas perfectamente simétricas entre habitaciones.

7. **Sustitución del *waypoint chasing* por un seguidor de polilínea con *lookahead***: el aumento de waypoints auxiliares para centrar la trayectoria en pasos estrechos mejoraba la geometría, pero degradaba la ejecución con cientos de micro-correcciones de rumbo, porque el controlador discreto original perseguía cada subgoal por separado con cuantización fija de 5° y 0,1 m. Se densifica ahora la ruta planificada a nivel de celda mediante rasterización de segmentos y se ejecuta con un *path follower* sobre esa polilínea densa. En cada iteración, el robot proyecta su pose sobre la ruta, selecciona un objetivo adelantado (*lookahead*) varios pasos por delante y solo aplica la primera acción discreta necesaria para aproximarse a ese punto. El *lookahead* se reduce automáticamente dentro de pasos estrechos y se mantiene más largo en espacio abierto. Además, las métricas de seguridad y la selección de metas de observación pasan a evaluarse sobre la misma polilínea densificada, de modo que planificación, validación y ejecución comparten por fin la misma representación geométrica del camino.

8. ***Lookahead* recortado por visibilidad local del camino**: el primer seguidor con *lookahead* seguía teniendo un fallo importante en pasillos con giro o marcos oblicuos: podía elegir como objetivo un punto varios pasos por delante que ya estaba al otro lado de una esquina no visible desde la pose actual. Como el controlador discreto solo orienta hacia ese objetivo y aplica la primera acción, el robot intentaba cortar la esquina y acababa proponiendo un avance con `front clearance = 0.0`. Se corrige haciendo que el *path follower* no use directamente el *lookahead* preferido, sino el punto más lejano de la polilínea densa que siga siendo alcanzable con línea de visión libre sobre el mapa de planificación dilatado. Así, dentro de cuellos de botella y giros cerrados, el seguidor deja de apuntar "a través de la pared" y se mantiene sobre el tramo realmente visible del camino.

9. **Etiquetas de llegada coherentes con el resultado real de la navegación**: en comandos de habitación se estaba mostrando "*Arrived: ...*" inmediatamente después de que terminara la ejecución del follower, incluso si el robot se había detenido antes del umbral y la comprobación final de habitación fallaba. Esto inducía a error durante la depuración

visual. Se corrige la interfaz para que el overlay solo muestre “Arrived” cuando el robot ha alcanzado de verdad la habitación objetivo; en caso contrario, ahora se muestra “Not reached” en navegación a habitaciones o “Stopped before goal” en aproximaciones a objetos.

10. **El follower vuelve a seguir la polilínea geométrica, no el raster denso**: el primer rediseño había reutilizado la trayectoria densificada a nivel de celda también como referencia de control. Eso resultó ser un error: la polilínea rasterizada introduce un patrón de “escalera” y el controlador discreto intentaba corregir rumbo en cada pequeño cambio de celda, disparando el número de acciones incluso cuando la ruta geométrica original ya era buena. Se separan ahora ambas representaciones: el follower usa como guía la polilínea de waypoints del planificador visgraph postprocesado, mientras que la versión densa se conserva solo para métricas, validación, escudo local y visualización. Además, el presupuesto máximo de ejecución pasa a escalar con la longitud geométrica real del camino y no con el preview antiguo de cientos de acciones discretas.

11. **Banda muerta angular para eliminar micro-correcciones de 5°**: incluso después de volver a seguir la polilínea geométrica, el controlador discreto seguía tendiendo a consumir pequeños giros antes de casi cada avance cuando el error angular hacia el siguiente objetivo era muy bajo. Eso no cambiaba de forma significativa la seguridad, pero sí alargaba innecesariamente la ejecución en pasos estrechos. Se añade una tolerancia angular en el follower: si el objetivo inmediato ya está alineado dentro de un margen moderado, se prioriza `move_forward` en lugar de corregir rumbo con otro giro discreto. La tolerancia es más amplia en zonas estrechas que en espacio abierto. El escudo local sigue validando cada avance, de modo que se reduce el número de pasos sin relajar la protección frente a colisiones.